

Trace-Driven Microservice Co-location through Temporal Resource Complementarity

TARO JOHO^{1,†1,a)} HANAKO SHORI^{2,b)} JIRO GAKKAI¹

Received: March 4, 2016, Accepted: August 1, 2016

Abstract: Resource-constrained cluster environments require microservices to share limited computing resources, while inappropriate co-location can cause severe resource contention. In practice, microservices participate unevenly in request processing, with different services being invoked at different frequencies. This uneven participation means that resource intensity alone does not necessarily reflect the actual pressure a service imposes under a given workload. Consequently, resource-based placement may misjudge resource competition and overlook potential co-location opportunities. Based on this insight, we propose a trace-driven microservice co-location method that combines measured CPU demand with trace-derived invocation characteristics to estimate workload-dependent resource pressure and guide placement decisions.

Keywords: Microservices, Kubernetes, Cloud-edge computing, Trace-driven scheduling, Resource reconfiguration

1. Introduction

Microservice architecture has become a widely adopted paradigm for cloud-native applications by decomposing monolithic systems into loosely coupled and independently deployable services [1]. Its adoption is demonstrated by a survey involving 1,502 respondents [2] and by large-scale production systems operated by companies such as Meta, Alibaba, and ByteDance [3]. For latency-sensitive workloads, performance degradation can cause SLO (Service-level Objective) violations and business losses [4], whereas preventing such violations through conservative overprovisioning increases resource waste and operating costs [2]. Therefore, efficient resource management is essential for balancing application performance and infrastructure cost.

Microservice applications contain many interconnected services, but each request typically invokes only a subset, as illustrated in Fig. 1a. In the Alibaba-derived μ Bench workloads [5], 398 of 652 application-service instances appear in fewer than 20% of traces, whereas 134 appear in at least 80% (Fig. 1b). This uneven participation is also observed in production workloads [3], [6], indicating that service pressure depends on both resource demand and invocation frequency. We therefore combine measured CPU demand with trace-derived invocation characteristics when evaluating co-location candidates.

Effective microservice co-location requires invocation information and resource contention to be considered before performance degradation occurs. Cao and Sharma use service-mesh telemetry to improve communication-aware container co-location

[7], whereas FIRM uses online telemetry and dynamic resource reprovisioning to mitigate contention-induced SLO violations [8]. These studies demonstrate the importance of communication affinity and resource pressure, but leave open how workload-dependent invocation characteristics can be combined with measured resource demand to identify unsafe co-location decisions in advance. Therefore, we propose a proactive trace-driven placement method that evaluates candidate configurations using CPU demand, invocation characteristics, and heterogeneous node capacity to prevent resource contention before deployment.

To address this gap, we propose a proactive trace-driven method for microservice co-location in resource-constrained cluster environments. By combining measured CPU demand with trace-derived invocation frequency, the method estimates workload-dependent service pressure and identifies co-location opportunities that resource-only approaches may overlook. By rejecting overload-prone placements before they are applied, the method aims to improve resource utilization without sacrificing application stability or execution efficiency. Its main advantages are summarized as follows:

- *Improved resource utilization:* The method co-locates services according to their workload-dependent pressure, reducing avoidable resource fragmentation and making more effective use of the available cluster capacity.
- *Higher execution stability:* Candidate placements are evaluated under multiple workload levels, allowing configurations that may concentrate excessive pressure on individual nodes to be rejected before they cause resource contention.
- *Improved execution efficiency:* A more balanced distribution of service pressure reduces CPU contention and request queuing, thereby helping microservice requests complete with lower latency and fewer failures.

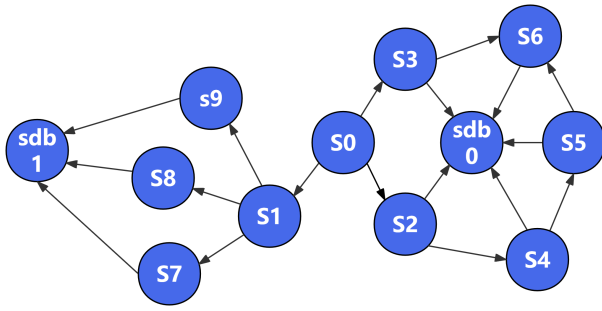
¹ Information Processing Society of Japan, Chiyoda, Tokyo 101-0062, Japan

² Johoshori University, Chiyoda, Tokyo 101-0062, Japan

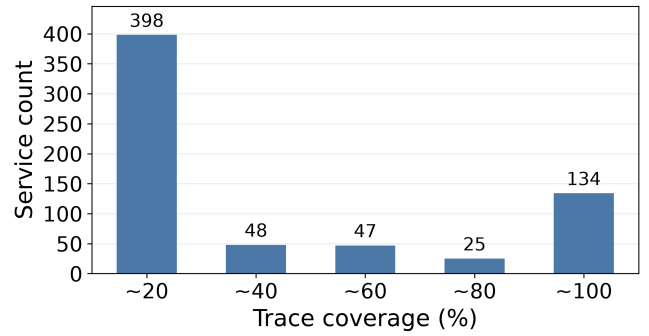
^{†1} Presently with Johoshori University

^{a)} joho.taro@ipsj.or.jp

^{b)} shori.hanako@johoshori-u.ac.jp



(a) Example microservice call structure.



(b) Distribution of service trace coverage.

Fig. 1: Service structure and trace-coverage characteristics of the workloads.

2. Related Work

Efficient management of microservices under limited and heterogeneous resources has long been recognized as an important research problem. Existing studies have approached this problem through microservice placement and co-location, SLO-aware resource provisioning, and interference prediction and mitigation. These directions respectively determine where services should run, how much resource each service should receive, and how contention among co-located workloads should be controlled. The following subsections review these directions and position our trace-driven co-location method within them.

2.1 Kubernetes Scheduling

In a resource-constrained cluster, the default Kubernetes scheduler assigns each newly created Pod to a feasible node based on its declared resource requirements and scheduling constraints [9]. It first filters nodes that cannot satisfy the Pod's CPU, memory, or placement requirements and then scores the remaining candidates to select a target node. This process ensures placement feasibility based on declared resource requests, but it does not represent the workload-dependent pressure imposed by a microservice during execution. Consequently, a placement considered feasible by the scheduler may still experience resource contention when multiple microservices are co-located.

Although resource-aware extensions improve the default Kubernetes scheduler, resource-level optimization alone remains insufficient for workload-sensitive microservice placement. **DRS** uses global resource monitoring and deep reinforcement learning to improve resource utilization and load balance [10], while Marchese and Tomarchio use real-time CPU, memory, network, and disk telemetry to support load-aware scheduling and rescheduling [11]. These methods provide a global view of cluster resources, but do not describe how an increasing request workload propagates through call paths and changes the invocation pressure imposed on individual services. Consequently, resource observations alone cannot evaluate such placement risks before workload changes appear in runtime metrics, motivating the use of trace-derived invocation information.

2.2 Co-location Opportunities

Microservice co-location refers to deploying multiple service instances on the same physical node to improve the utilization of shared computing resources. Heracles demonstrates the feasibility of this idea by safely sharing CPU, memory, cache, and network resources between latency-critical and best-effort workloads [12]. At the microservice level, FIRM shows that resource sharing can produce service-level contention and that the responsible service and resource type must be identified before resources are dynamically reprovisioned [8]. These studies establish both sides of the co-location problem: sharing idle capacity can improve resource efficiency, but inappropriate service combinations can degrade application performance.

Recent research has therefore moved from runtime interference mitigation toward selecting suitable service combinations during placement. Adeppady et al. propose iPlace to group microservices with diverse resource competition characteristics, thereby reducing interference and the number of required servers in resource-constrained cloud systems [13]. Yang et al. further show that workload combinations with similar aggregate resource consumption can produce different interference levels and use composition-specific interference scores to guide proactive scheduling [14]. These findings indicate that static CPU quantities alone cannot determine whether a co-location is safe, motivating our combination of measured CPU demand and trace-derived invocation frequency to estimate workload-dependent pressure before applying a candidate placement.

2.3 Prediction-based Resource Management

Prediction-based resource management enables cluster managers to evaluate the future consequences of resource decisions before applying them. Sinan predicts near-future tail latency and longer-term QoS violation probability under candidate per-service CPU allocations and periodically selects a configuration that satisfies the end-to-end latency target [15]. By evaluating the expected effect of each configuration, this approach can avoid overly conservative provisioning as well as resource reductions that would later produce queue accumulation and cascading QoS violations. Sinan therefore establishes prediction as a practical mechanism for proactively adjusting microservice resources.

Recent research has extended this direction from QoS-aware

allocation toward interference-aware management of co-located microservices. C-Koordinator predicts CPI from node-, Pod-, and application-level metrics and uses the predicted interference severity to guide CPU suppression or Pod eviction in large-scale co-located clusters [4]. Whereas C-Koordinator focuses on forecasting emerging hardware-level interference and mitigating it during cluster operation, our method combines measured CPU demand with trace-derived invocation characteristics to estimate workload-dependent service pressure and evaluate candidate co-locations before they cause contention. Our work therefore complements runtime interference prediction by introducing request-path information into proactive microservice placement.

2.4 Trace-driven Placement

Distributed traces can be transformed into application-level information for cluster management rather than serving only as execution records. MicroRCA reconstructs service dependency graphs from traces and combines them with resource metrics to localize the microservice and resource responsible for a performance problem [16], while Sage uses trace-derived causal models and counterfactual reasoning to identify the causes of QoS violations [17]. These studies demonstrate that traces reveal how performance symptoms propagate across dependent services and can therefore indicate where resource intervention should be directed. However, their primary purpose is performance diagnosis and corrective reasoning rather than evaluating the resource pressure of alternative placements before they are applied.

More closely related studies use trace-derived information for provisioning or placement. Colarusso et al. construct workload-dependent graphs and co-locate strongly interacting services to improve communication locality [1], whereas Morphis predicts recurring call-graph patterns to support SLO-aware CPU provisioning [3]. The former does not explicitly account for node capacity, while the latter focuses on CPU and replica allocation rather than placement. In contrast, our method combines invocation frequency with measured CPU demand to estimate the aggregate pressure of co-located services on heterogeneous nodes, thereby identifying placements that remain safe as request volume increases.

3. Trace-Driven Co-location Scheduler

The proposed framework interprets a distributed trace not only as a service dependency graph, but also as a temporal record of how computing resources are occupied during request processing. It first identifies the execution intervals and invocation multiplicities of individual services from traces, and then combines this information with measured CPU demand to estimate how service pressure changes as request concurrency increases. Based on these estimates, the framework generates candidate co-locations and evaluates whether the remaining capacity of each node can safely accommodate the resulting aggregate pressure. Figure 2 illustrates the overall workflow, consisting of trace-derived window characterization, workload-dependent pressure estimation, candidate placement generation, and load-growth-aware filtering.

3.1 Co-location Windows

Distributed traces reveal potential temporal headroom between service executions. As illustrated in Fig. 3, the CPU-active phase of S1 does not fully overlap with the main execution phases of S2, S3, and S4. Placing services with such complementary execution intervals on the same node could improve cluster utilization without imposing their full CPU demands simultaneously. Prior work provides evidence that this trace-visible headroom can be exploited.

Zhu et al. use per-request execution footprints and critical-path analysis to identify performance-sensitive services and harvest idle resources for co-scheduling [18]. Based on this interpretation, we define the trace-visible temporal headroom as a candidate co-location window. However, it cannot be treated as directly available capacity because an interval observed from one request may change under concurrent workloads.

As request concurrency increases, overlapping executions and queued requests may fill the observed window. Nodens demonstrates the effects of load dynamics and execution blocking [19], while Morphis shows that invocation-pattern frequency determines the realized service load [3]. Therefore, our method combines trace-derived invocation characteristics with measured CPU demand and evaluates candidate co-locations under increasing request volumes, rather than directly relying on idle intervals from individual traces.

3.2 Trace-Driven Framework

The proposed framework performs proactive microservice co-location through a periodic observation and adjustment cycle. As shown in Fig. 2, each cycle consists of telemetry collection, trace-metric evaluation, and deployment-based rescheduling. Separating evaluation from execution allows alternative placements to be examined under projected workloads before any running service is moved. The following subsections describe these three stages in their execution order.

3.2.1 Telemetry Collection

Each adjustment cycle begins by collecting application- and resource-level information from the running cluster. Istio sidecars record distributed traces in Jaeger, including request paths and service invocation counts, while Kubelet/cAdvisor and Prometheus provide container-level CPU measurements; the current placement and node capacities are obtained through Kubernetes. These data connect the invocation characteristics of each service with its measured CPU demand and current execution environment. The collected observations are subsequently used to evaluate alternative co-location plans.

3.2.2 Scoring and Evaluation

The framework jointly analyzes trace and metric information to estimate the workload-dependent pressure of each service. Measured CPU demand is combined with trace-derived invocation frequency and projected request volume, after which alternative service-to-node mappings are evaluated against the available capacity of each node. Infeasible candidates are filtered out, while the remaining plans are ranked according to their estimated overload area and resource balance, with the current deployment retained when no candidate provides an improvement. The highest-

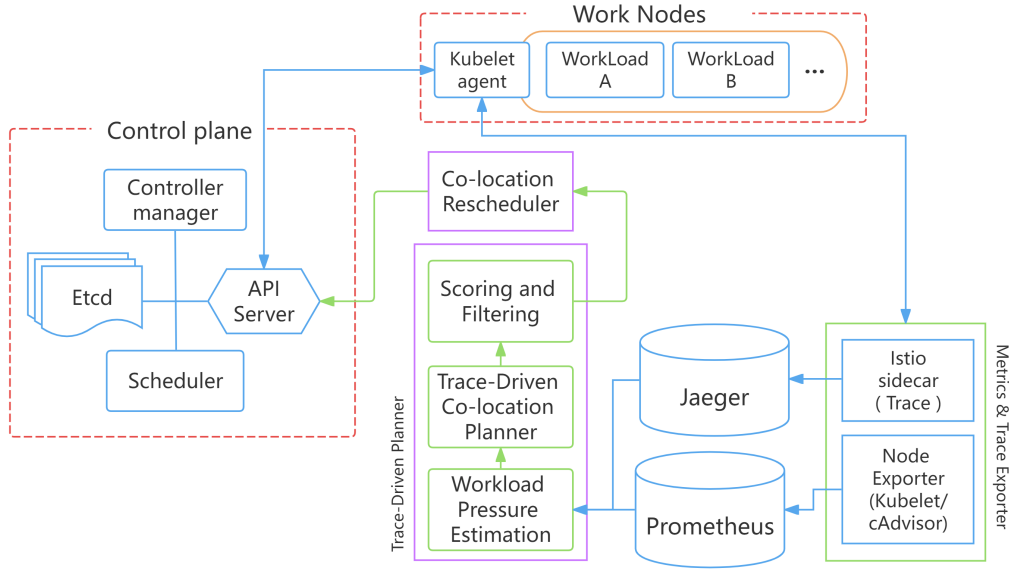


Fig. 2: Architecture of the proposed trace-guided scheduling support method.

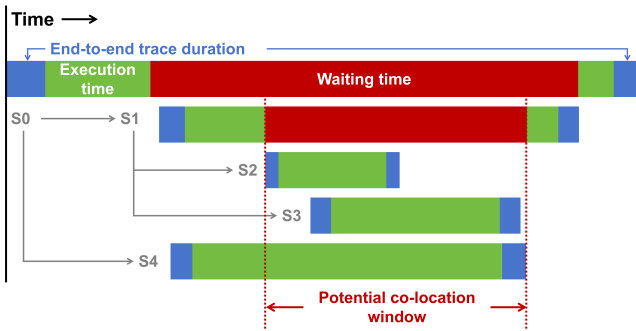


Fig. 3: Illustration of a potential co-location window within a request trace.

ranked feasible plan is then passed to the co-location rescheduler.

3.2.3 Deployment-Based Rescheduling

The co-location rescheduler applies the selected plan by updating the placement constraints of only the affected Kubernetes Deployments through the API server. It then triggers the Deployment rolling-update mechanism, allowing Kubernetes controllers to create replacement Pods and the default scheduler to place them on the selected nodes before the old Pods are terminated under the configured readiness and availability policy. Services unaffected by the selected plan continue running without redeployment, reducing unnecessary disruption. The updated placement becomes the cluster state used in the next observation cycle.

3.3 Methodology

The proposed method characterizes each service using three parameters: **baseline-adjusted CPU usage**, **trace-derived calls per entry request**, and **the CPU request configured for its Pods**. These parameters are calculated because measured CPU usage captures observed processing demand, the call factor reflects uneven service participation in end-to-end requests, and CPU requests determine whether a placement is feasible in Kubernetes. By ag-

gregating them for each candidate node, the framework can balance processing demand and invocation intensity, filter placements that exceed reserved capacity, and estimate CPU overload as the workload increases. The following paragraphs define these parameters and explain how they are used to generate, score, and select co-location plans.

3.3.1 Baseline-Adjusted CPU Usage

The framework derives the CPU demand of each service replica from Pod-level measurements collected during a Prometheus observation window of duration Δt . Let C_p denote the cumulative CPU time consumed by Pod p , b_p its average idle CPU usage, and $\mathcal{P}_s$ the set of observed Pods belonging to service s . The baseline-adjusted CPU demand assigned to replica j of service s is calculated as

$$u_{s,j} = \frac{\sum_{p \in \mathcal{P}_s} \max(0, C_p - b_p \Delta t)}{r_s \Delta t}, \quad j = 1, \dots, r_s, \quad (1)$$

where r_s is the configured number of replicas. The resulting value represents the average CPU cores consumed above the idle baseline, assuming that the measured service-level demand is distributed evenly among its replicas. Because C_p accumulates the CPU time generated by every activation of Pod p during the observation window, normalization by Δt jointly reflects both the intensity of its CPU consumption and how frequently or persistently that consumption occurs. A service that consumes substantial CPU only during a small number of short executions therefore produces a lower sustained CPU demand, whereas a frequently or continuously active service can retain a significant value even when its instantaneous CPU consumption is relatively small.

Consequently, $u_{s,j}$ represents the realized CPU pressure of the service under the observed workload rather than its isolated peak resource intensity. A Pod-specific baseline is used when available; otherwise, the mean per-Pod baseline of the corresponding service is adopted. This implicit frequency effect does not re-

veal how the service participates in individual request paths, so the following analysis additionally derives an explicit call factor from distributed traces.

3.3.2 Trace-Derived Call Factor

The trace analyzer processes the Jaeger JSON files and counts the server-side spans associated with each microservice. Client-side spans are excluded to avoid counting the same inter-service invocation twice, while repeated server-side spans are retained because they represent multiple executions of the service within an end-to-end request. Let N_s denote the total number of calls to service s and N_e the total number of calls to the entry service e . The trace-derived call factor is calculated as

$$F_s = \frac{N_s}{N_e}. \quad (2)$$

The resulting value represents the average number of calls to service s generated by one entry request: $F_s < 1$ indicates that the service participates in only some request paths, whereas $F_s > 1$ indicates that it may be invoked repeatedly within a request. The analyzer outputs both N_s and F_s for every observed service, and the placement model subsequently distributes the service-level factor evenly among its configured replicas.

3.3.3 Candidate Placement Generation and Selection

Each replica p is represented by its measured CPU usage u_p , trace-derived call factor f_p , and CPU request q_p . For a placement x , these values are aggregated as U_n , F_n , and Q_n on each node n . A placement is feasible when $Q_n \leq C_n$ for every node. Candidate generation minimizes

$$B(x) = V(U_n/C_n) + V(F_n/K_n) + V(Q_n/C_n), \quad (3)$$

where K_n is the number of replicas assigned to node n and $V(\cdot)$ denotes relative dispersion across nodes. This objective balances measured CPU load, invocation intensity per replica, and reserved CPU.

Algorithm 1 summarizes the placement process. Randomized greedy assignment first constructs a feasible placement, which is subsequently refined through replica relocation and exchange. Such local-search operations provide a practical trade-off between placement quality and search cost [20]. The process is repeated 300 times to obtain distinct candidate placements.

Each candidate is evaluated under base and hotspot workload growth, following the motivation of service placement under uncertain demand [21]. The overload area and final score are

$$A(L) = \sum_n \int_1^4 [\max(0, kL_n - C_n)]^\rho dk, \quad (4)$$

$$R(x) = A(U) + \lambda \max(0, A(U^{\text{hot}}) - A(U)),$$

where $\rho = 2$ penalizes severe overload and $\lambda = 1$ weights the additional hotspot penalty. Integrating over $k \in [1, 4]$ considers both the onset and severity of overload. The candidate with the smallest $R(x)$ is passed to the co-location rescheduler, while $B(x)$ is used to break ties.

4. Evaluation

4.1 Experimental Setup

We constructed an emulated cloud-edge testbed using KVM

Algorithm 1 Proposed candidate generation and selection

Require: Replicas P , nodes N , restarts $M = 300$

Ensure: Selected placement x^*

- 1: Select $H = 3$ CPU-intensive services with positive, non-maximum call factors as $\mathcal{H}$
- 2: $C \leftarrow \emptyset$
- 3: **for** $i = 1$ to M **do**
- 4: Order replicas by $u_p/C_{\max} + f_p + q_p$ with randomized tie-breaking
- 5: Construct feasible placement x by assigning each replica to the node minimizing the partial $B(x)$
- 6: Refine x through feasible replica relocation and exchange
- 7: Compute base node load U
- 8: Compute hotspot load U^{hot} by multiplying the demand of services in $\mathcal{H}$ by $\gamma = 2$
- 9: Compute $(R(x), B(x))$ and store the distinct candidate
- 10: **end for**
- 11: Rank candidates lexicographically by $(R(x), B(x))$
- 12: Retain the five best candidates
- 13: **return** the first candidate as x^*

Table 1: Configuration of the experimental cluster.

Node	Role	vCPU	Memory (GiB)	Disk (GiB)
client-0	Control/client	2	4	40
cloud-0	Cloud worker	8	16	50
edge-1	Edge worker	4	8	30
edge-2	Edge worker	2	4	30
edge-3	Edge worker	2	4	30

Table 2: Methods used in the comparison.

Method	Placement basis	Evaluation purpose
Default	Original placement	Practical baseline
Greedy	Measured CPU/s	CPU-aware baseline
Proposed	CPU/s and trace factor	Complete method

virtual machines on a physical server equipped with two Intel Xeon E5-2650 v2 processors and approximately 60 GiB of memory. The testbed runs MicroK8s and consists of one control/client node and four worker nodes, as summarized in Table 1.

The application Pods are deployed only on the four worker nodes, while client-0 hosts the control-plane functions and generates the test workloads. The heterogeneous worker capacities, with a CPU ratio of 8 : 4 : 2 : 2, represent one resource-rich cloud node and three resource-constrained edge nodes. Istio collects distributed traces, while Prometheus records Pod- and node-level CPU metrics.

4.2 Comparison Design

To distinguish the contribution of CPU awareness from that of trace-aware overload selection, we compare **Default**, **Greedy**, and **Proposed**. This progressive design follows established evaluations that compare an operational baseline, a resource-driven heuristic, and the complete method [8], [15]. Greedy is included as the intermediate baseline because greedy and local-search heuristics are commonly used in cluster and cloud-edge scheduling to obtain feasible placements with moderate search overhead [20], [22], [23]. Table 2 summarizes the role of each method.

Default retains the original Kubernetes placement. Greedy

Table 3: Preliminary request completion results.

Method	Success	Error (%)	504	Mean (s)
500 requests				
Default	412	17.6	61	179.4
Greedy	474	5.2	1	136.4
Proposed	477	4.6	0	129.4
1,000 requests				
Greedy	279	72.1	706	177.6
Proposed	510	49.0	480	172.2

generates 300 feasible candidates using measured CPU/s and selects the placement with the smallest normalized CPU dispersion. Proposed uses the same randomized greedy framework, feasibility constraints, and search budget, but additionally incorporates the trace call factor and selects the placement with the smallest robust overload score. Because the search is randomized, their candidate sets need not be identical. All methods use the same cluster capacity, replica counts, resource constraints, workload composition, and request sequence. Default versus Greedy evaluates the benefit of CPU-aware placement, Greedy versus Proposed assesses the additional contribution of trace-aware selection, and Default versus Proposed evaluates the complete method. The following subsection compares their request success rates and P95/P99 latency under the same workload levels, treating success rate as the primary metric under high load.

4.3 Results

Table 3 summarizes request completion and errors, while Fig. 4 shows the latency distributions of successfully completed requests. With 500 requests, Proposed achieved a success rate of 95.4%, compared with 94.8% for Greedy and 82.4% for Default. Relative to Greedy, Proposed reduced the mean latency from 136.4 s to 129.4 s and the median from 130.9 s to 123.0 s, corresponding to reductions of 5.2% and 6.1%, respectively. Its P99 increased slightly from 263.2 s to 269.4 s, indicating that the improvement at moderate load was concentrated around the center of the distribution rather than across the entire tail.

The advantage of Proposed became more apparent with 1,000 requests. It completed 510 requests, compared with 279 for Greedy, reducing the error rate from 72.1% to 49.0% and HTTP 504 errors from 706 to 480. Despite completing substantially more requests, Proposed also reduced the mean latency by 3.1%, the median from 186.9 s to 183.6 s, and P99 from 297.7 s to 295.3 s. Because both P99 values approached the 300 s timeout boundary, their small difference does not indicate equivalent request-processing capacity. Instead, the substantially higher success rate provides the stronger evidence that trace-aware placement improves overload tolerance, rather than uniformly reducing every latency percentile.

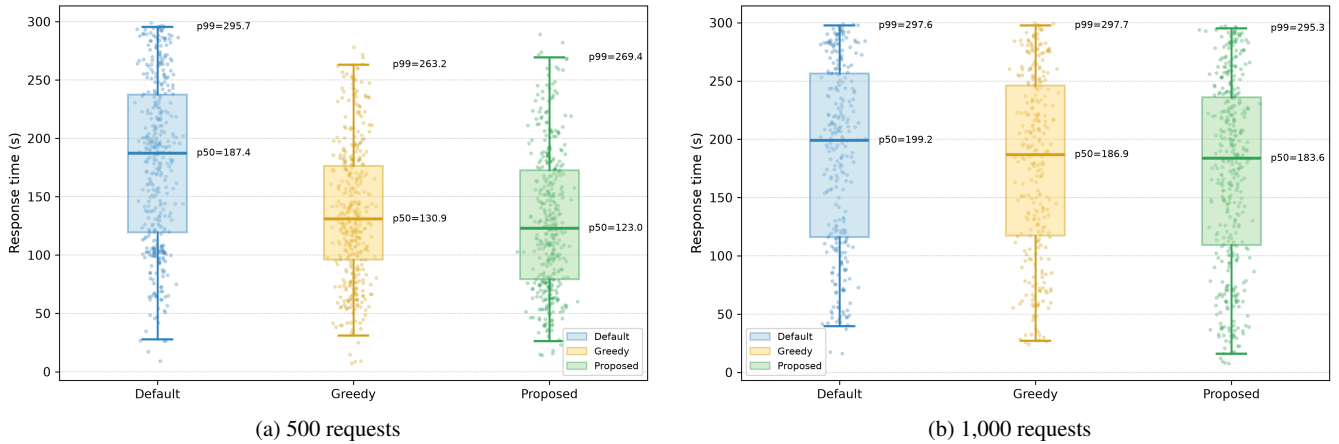


Fig. 4: Latency distributions under different workload levels. Only successfully completed requests are included.

5. Discussion

Acknowledgments We wrote this article based on the guideline for A4 landscape layout. We are grateful to Prof. Hiroshi Nakashima from Kyoto University, for his valuable comments on making a class-file, and his consent to usage of BiBTeX files. We are also very thankful to the editorial committee for their contributions in writing the guideline for the A4 landscape layout.

References

- [1] Colarusso, C., De Caro, A., Falco, I., Goglia, L. and Zimeo, E.: A Distributed Tracing Pipeline for Improving Locality Awareness of Microservices Applications, *Software: Practice and Experience*, Vol. 54, No. 6, pp. 1118–1140 (online), DOI: 10.1002/spe.3317 (2024).
- [2] Park, J., Choi, B., Lee, C. and Han, D.: GRAF: A Graph Neural Network Based Proactive Resource Allocation Framework for SLO-Oriented Microservices, *Proceedings of the 17th International Conference on Emerging Networking Experiments and Technologies*, ACM, pp. 154–167 (online), DOI: 10.1145/3485983.3494866 (2021).
- [3] Tang, Y., Zhao, H., Lu, C., Zhang, Y., Chow, K., Deng, S. and Shi, R.: Morphis: SLO-Aware Resource Scheduling for Microservices with Time-Varying Call Graphs, arXiv preprint arXiv:2602.01044 (2026). Version 2.
- [4] Song, S., Xu, M., Zhang, Z., Gao, C., Zeng, F., Ding, Y., Ye, K. and Xu, C.: C-Koordinator: Interference-Aware Management for Large-Scale and Co-Located Microservice Clusters, *Software: Practice and Experience*, Vol. 56, No. 5, pp. 506–524 (online), DOI: 10.1002/spe.70059 (2026).
- [5] Detti, A., Funari, L. and Petrucci, L.: μ Bench: An Open-Source Factory of Benchmark Microservice Applications, *IEEE Transactions on Parallel and Distributed Systems*, Vol. 34, No. 3, pp. 968–980 (online), DOI: 10.1109/TPDS.2023.3236447 (2023).
- [6] Luo, S., Xu, H., Lu, C., Ye, K., Xu, G., Zhang, L., Ding, Y., He, J. and Xu, C.: Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis, *Proceedings of the ACM Symposium on Cloud Computing*, ACM, pp. 412–426 (online), DOI: 10.1145/3472883.3487003 (2021).
- [7] Cao, L. and Sharma, P.: Co-Locating Containerized Workload Using Service Mesh Telemetry, *Proceedings of the 17th International Conference on Emerging Networking Experiments and Technologies*, ACM, pp. 168–174 (online), DOI: 10.1145/3485983.3494867 (2021).
- [8] Qiu, H., Banerjee, S. S., Jha, S., Kalbarczyk, Z. T. and Iyer, R. K.: FIRM: An Intelligent Fine-Grained Resource Management Framework for SLO-Oriented Microservices, *14th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, pp. 805–825 (2020).
- [9] Kubernetes: Kubernetes Scheduler, <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/> (2024). Accessed: 2026-05-14.
- [10] Jian, Z., Xie, X., Fang, Y., Jiang, Y., Lu, Y., Dash, A., Li, T. and Wang, G.: DRS: A Deep Reinforcement Learning Enhanced Kubernetes Scheduler for Microservice-Based System, *Software: Practice and Experience*, Vol. 54, No. 10, pp. 2102–2126 (online), DOI: 10.1002/spe.3284 (2024).
- [11] Marchese, A. and Tomarchio, O.: Enhancing the Kubernetes Platform with a Load-Aware Orchestration Strategy, *SN Computer Science*, Vol. 6, p. 224 (online), DOI: 10.1007/s42979-025-03712-z (2025).
- [12] Lo, D., Cheng, L., Govindaraju, R., Ranganathan, P. and Kozyrakis, C.: Heracles: Improving Resource Efficiency at Scale, *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, ACM, pp. 450–460 (online), DOI: 10.1145/2749469.2749475 (2015).
- [13] Adeppady, M., Giaccone, P., Karl, H. and Chiasserini, C. F.: Reducing Microservices Interference and Deployment Time in Resource-Constrained Cloud Systems, *IEEE Transactions on Network and Service Management*, Vol. 20, No. 3, pp. 3135–3147 (online), DOI: 10.1109/TNSM.2023.3235710 (2023).
- [14] Yang, D., Zheng, K., Qian, S., Hua, Q., Zhang, K., Cao, J. and Xue, G.: Mitigating Interference of Microservices with a Scoring Mechanism in Large-Scale Clusters, *The Journal of Supercomputing*, Vol. 81, No. 1, p. 104 (online), DOI: 10.1007/s11227-024-06534-7 (2025).
- [15] Zhang, Y., Hua, W., Zhou, Z., Suh, G. E. and Delimitrou, C.: Sinan: ML-Based and QoS-Aware Resource Management for Cloud Microservices, *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ACM, pp. 167–181 (online), DOI: 10.1145/3445814.3446693 (2021).
- [16] Wu, L., Tordsson, J., Elmroth, E. and Kao, O.: MicroRCA: Root Cause Localization of Performance Issues in Microservices, *NOMS 2020–2020 IEEE/IFIP Network Operations and Management Symposium*, IEEE, pp. 1–9 (online), DOI: 10.1109/NOMS47738.2020.9110353 (2020).
- [17] Gan, Y., Liang, M., Dev, S., Lo, D. and Delimitrou, C.: Sage: Practical and Scalable ML-Driven Performance Debugging in Microservices, *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ACM, pp. 135–151 (online), DOI: 10.1145/3445814.3446700 (2021).
- [18] Zhu, J., Yang, R., Sun, X., Wo, T., Hu, C., Peng, H., Xiao, J., Zomaya, A. Y. and Xu, J.: QoS-Aware Co-Scheduling for Distributed Long-Running Applications on Shared Clusters, *IEEE Transactions on Parallel and Distributed Systems*, Vol. 33, No. 12, pp. 4818–4834 (online), DOI: 10.1109/TPDS.2022.3202493 (2022).
- [19] Shi, J., Zhang, H., Tong, Z., Chen, Q., Fu, K. and Guo, M.: Nodens: Enabling Resource Efficient and Fast QoS Recovery of Dynamic Microservice Applications in Datacenters, *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, USENIX Association, pp. 403–417 (2023).
- [20] Kumar, N., Ruiz, P. M., Menon, V., Kabiljo, I., Pundir, M., Newell, A., Lee, D., Wang, L. and Tang, C.: Optimizing Resource Allocation in Hyperscale Datacenters: Scalability, Usability, and Experiences, *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, USENIX Association, pp. 507–528 (2024).
- [21] Nguyen, D. T., Nguyen, H., Trieu, N. and Bhargava, V. K.: Two-Stage Robust Edge Service Placement and Sizing under Demand Uncertainty, *IEEE Internet of Things Journal*, Vol. 9, No. 2, pp. 1560–1574 (online), DOI: 10.1109/JIOT.2021.3090442 (2022).
- [22] Suresh, L., Loff, J., Kalim, F., Jyothis, S. A., Narodytska, N., Ryzhyk, L., Gamage, S., Oki, B., Jain, P. and Gasch, M.: Building Scalable and Flexible Cluster Managers Using Declarative Programming, *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, USENIX Association, pp. 827–844 (2020).
- [23] Tang, B., Luo, J., Obaidat, M. S. and Vijayakumar, P.: Container-

Based Task Scheduling in Cloud-Edge Collaborative Environment Using Priority-Aware Greedy Strategy, *Cluster Computing*, Vol. 26, pp. 3689–3705 (online), DOI: 10.1007/s10586-022-03765-2 (2023).